

CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference

A Synopsis Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology

in

Information Technology

by

Shubhankar Vishnu Patil 23AIFT1101087

Vivek Dnyaneshwar Joshi 23AIFT1101076

Yash Suresh Kodgirwar 23AIFT1108097

Guided by:

Mr. Nirmalya Mukhopadhyay

Asst. Prof., Dept. of IT

Mr. Kunal Kundu

Asst. Prof., Dept. of IT



Department of Information Technology

G H Raison College of Engineering and Management, Pune

Winter 2026

Academic Year 2026–27

Declaration

We hereby declare that the project synopsis entitled **CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference** submitted for the partial fulfillment of the requirements for the degree of Bachelor of Technology in Information Technology is our own work and has not been submitted previously by us or anyone else for the award of any degree, diploma, or other similar titles.

We further declare that the information presented in this synopsis is the result of our own work, and all sources of information have been acknowledged by proper references.

Submitted by:

Shubhankar Vishnu Patil
23AIFT1101087

Vivek Dnyaneshwar Joshi
23AIFT1101076

Yash Suresh Kodgirwar
23AIFT1108097

Date: _____

Abstract

Modern software development practices increasingly rely on continuous integration and continuous delivery (CI/CD) to ship code rapidly. However, automated security review at the pull request stage remains limited, relying either on rigid static analysis tools or homogeneous multi-agent systems whose findings offer little diagnostic depth. In this project we will present the design and implementation of CodeSheriff, an agentic security auditor that will assist developers in identifying and remediating vulnerabilities through multi-modal analysis and controlled, evidence-based automation. We will integrate proposed system directly with GitHub and will automatically analyze every pull request as it is opened. Each pull request will be processed by multiple specialist agents, including structural analysis using graph-based learning on abstract syntax trees, semantic reasoning using a fine-tuned language model, and contextual analysis using historical vulnerability data. In addition, we will apply Software Fault Isolation (SFI) principles as part of a confidential execution layer, so that pull request code can be observed running in a controlled, isolated environment as a further source of evidence. A Bayesian inference engine will fuse the evidence from all active agents to produce a calibrated verdict along with a measure of confidence in that verdict, enabling more reliable and interpretable decision-making than simple voting-based approaches. The system will use a patch generation and verification module to automatically suggest corrective code changes and confirm their effectiveness before presenting the results to the developer. The system will tailor its results, findings, and recommendations to the details of each pull request, and will share them through both GitHub comments and a web-based dashboard, making the results easier to understand and use. Through this SFI-backed execution layer, the system will also aim to guard the CI/CD workflow itself against compromise, not just the code passing through it. The system will aim to reduce contemporary security review effort and will provide structured, statistically grounded assistance, minimizing false positives and improving the accuracy of vulnerability detection in real-world development processes. In this project we will demonstrate the practical application of agentic, multi-modal automation, SFI-based confidential execution, and probabilistic evidence fusion in software security.

Contents

Abstract	ii
1 Introduction	1
1.1 Motivation	2
1.2 Objectives of the Project	2
2 Literature Review	3
3 Problem Statement	9
4 Proposed Methodology	10
4.1 System Architecture	10
4.2 Tools & Technology	12
5 Project Planning and Implementation	13
6 Expected Outcome	15
7 Conclusion	16
References	17

1 Introduction

Modern software development continues to rely heavily on CI/CD pipelines to deliver software quickly and efficiently. Platforms such as GitHub have become an integral part of collaborative software development, where developers contribute, review, and merge code through pull requests. Although this workflow improves development speed and collaboration, ensuring that every pull request is secure remains a challenging task. Security reviews often require experienced developers to identify potential vulnerabilities, understand their impact, and verify whether the reported issues are genuine. As software projects continue to grow in size and complexity, developers also face challenges such as inconsistent code reviews, alert fatigue caused by noisy security tools, and limited time for thorough manual analysis.

To address these challenges, this project proposes an intelligent security auditing framework capable of performing comprehensive and automated vulnerability analysis on GitHub pull requests. Most existing security tools rely either on traditional static analysis, which is effective only for predefined vulnerability patterns, or on single-perspective analysis techniques that may overlook important security issues. Since these approaches depend on a single analysis method, they often generate false positives or fail to detect complex vulnerabilities. As modern software systems continue to evolve, there is a growing need for a more reliable and scalable solution that combines multiple perspectives while reducing the manual effort required during code review.

The proposed system, CodeSheriff, is designed as an agentic security auditing platform that combines multiple specialized AI agents with a Bayesian evidence fusion mechanism. Each pull request is analyzed independently from structural, semantic, and contextual perspectives, allowing the system to gather diverse evidence before making a security decision. Instead of presenting multiple isolated results, the Bayesian Inference Engine combines the evidence into a single confidence-aware vulnerability assessment. In addition to identifying security issues, the system also generates patch recommendations and verifies the suggested fixes before presenting them to the developer, helping reduce both review effort and remediation time.

A key feature of the proposed system is its Bayesian Inference Engine, which provides a principled way of combining evidence collected from different analysis agents. Rather than relying on simple majority voting or averaging the outputs of multiple agents, the engine treats each agent’s prediction as statistical evidence and computes the probability that a vulnerability exists. This probabilistic approach produces more reliable and explainable security decisions while reducing the influence of individual agent limitations. As a result, the system delivers confidence-aware vulnerability reports that are easier for developers and security reviewers to understand and trust.

1.1 Motivation

The motivation for this project comes from a clear gap in how CI/CD pipelines handle security today: existing tools either rely on rigid static analyzers that miss context-dependent vulnerabilities, or use multiple LLM agents that all read code the same way, making their "disagreement" meaningless. As developers who've relied on these pipelines ourselves, this project is motivated by the desire to explore agentic system design, Bayesian reasoning under uncertainty, and confidential computing, while building a realistic system that demonstrates how intelligent, multi-modal evidence fusion can make vulnerability detection both more accurate and more trustworthy, while also hardening the pipeline itself against compromise.

1.2 Objectives of the Project

- To design and implement a multi-agent AI framework that automatically audits GitHub Pull Requests for security vulnerabilities as part of the CI/CD pipeline.
- To improve vulnerability detection coverage by analyzing code across multiple complementary modalities such as structural, semantic, and historical/contextual, rather than relying on a single analysis method.
- To reduce false-positive rates in automated vulnerability detection through a collaborative, confidence-weighted decision-making mechanism among specialist agents, including a formal measure of inter-agent disagreement as an additional risk signal.
- To deliver actionable, developer-facing outcomes in plain-English vulnerability explanations, severity ratings, and auto-generated patch suggestions directly within the GitHub pull request workflow, enabling continuous and secure software development.

2 Literature Review

Recent advancements in Deep Learning, Large Language Models, and System Security have significantly improved automated software vulnerability detection. Several research works have focused on code-based vulnerability identification using Graph Neural Networks, transformer architectures, and multi-agent LLM frameworks. The following literature review summarizes major research contributions in this domain.

Table 1: Literature Review Summary

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[1]	P.-C. Huang, C.-H. Mao, and C.-M. Lai, “Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification,” IEEE Access, 2026.	Integrates structured host telemetry, Retrieval-Augmented Generation (RAG), FAISS-based semantic retrieval, and LLM reasoning to detect vulnerabilities and assign context-aware L0–L4 risk levels using dynamic environmental analysis.	The framework achieved up to 0.79 F1-score for CVE detection and over 90% risk classification accuracy within ± 1 level, outperforming static scanners through semantic reasoning and contextual threat intelligence integration.	Extending support to cloud-native and Kubernetes environments, correlating multi-host telemetry, improving zero-day vulnerability detection, integrating advanced LLM cybersecurity frameworks, and enabling large-scale distributed enterprise deployment.
[2]	R. A. Almutairi and N. Abbadeni, “Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code,” IEEE Access, 2026.	Integrates a transformer encoder (CodeBERT or GraphCodeBERT) with BiLSTM and CNN pooling layers.	Achieved a macro-averaged F1-score of 96.93%, outperforming existing Python-based baselines by 11.16%.	Expand scope to additional security smells and statically typed languages.

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[3]	G. Antal, L. 'Ertekes, N. Szolnoki, and P. Heged"us, "Evaluating retrieval-augmented gen-eration for llm-based vulnerability detection: An empirical study on real-world java vul-nerabilities," IEEE Access, 2026.	Evaluated LLMs (including GPT-4o and Gemini) on 72 vulnerable-fixed pairs from the Vul4J dataset.	RAG improved performance, reducing false positives by 12.54% and in-creasing accuracy by 4.95%.	Investigate multi-language evaluations and combining RAG with model fine-tuning.
[4]	B. K. E. Abbasi and A. Shameli-Sendi, "Lever-aging key source code metrics for predict-ing software vulnerabilities with machine learning," IEEE Access, 2026. [4]	Parsing source code with a fuzzy analyzer to construct a Code Property Graph, from which 14 human-interpretable metrics are ex-tracted for each function.	Random Forest achieved superior performance with an accuracy of 0.96. Using SMOTE-Tomek improved accu-racy by 13.6%.	Extend the proposed frame-work to other programming languages such as Python and Java.Combine predictive models with autonomous, real-time security systems.
[5]	H. Gao, D. Zhang, and L. Chen, "Scout: Llm-driven vulnerability de-tection via interactive repository exploration," IEEE Access, 2026.	Operates in an offline phase to construct a two-layer reposi-tory architecture. Online, it ini-tializes a local subgraph cen-tered around the target function.	Achieved state-of-the-art results on the PrimeVul benchmark with 57.93% accuracy and the lowest false positive rate (41.84%) without fine-tuning.	Replicating evaluation on al-ternative datasets and extending tree-sitter parsing to support lan-guages like Java and Python.

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[6]	H. Su, Z. Xu, Y. Zhang, and Q. Tan, "Source code vulnerability detection based on deep learning: a review," <i>Cybersecurity</i> , vol. 9, no. 1, p. 2, 2026.	Maps research across a three-phase analytical framework to evaluate benchmark datasets and examine deep learning model structures.	Field shifting to multimodal hybrid and advanced LLM-RAG architectures. Real-world datasets heavily skew toward C/C++ and suffer from class imbalance.	Developing automated, multi-language dataset curation frameworks. Advancing deep modality fusion through cross-modal attention.
[7]	L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, "A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models," <i>IEEE Access</i> , 2025.	This systematic review uses the PRISMA framework to analyze 208 studies regarding large language models for vulnerability analysis.	Most studies dominantly focus on vulnerability detection (91.3%), heavily leaving repair (11.1%) and textual explanation (5.3%) underexplored.	Create robust evaluation frameworks, standardize benchmarks, and expand coverage to underrepresented programming languages.
[8]	Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, "Advanced smart contract vulnerability detection via llm-powered multi-agent systems," <i>IEEE Transactions on Software Engineering</i> , 2025.	A collaborative multi-agent conversation framework utilizing inception prompts, role swapping, and a dynamic buffer-of-thought mechanism.	The framework achieved 98% accuracy on common vulnerabilities, identifying 12 out of 13 CVEs efficiently.	Integrate more advanced language models, adaptive memory management, and hybrid formal verification techniques.

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[9]	A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, “Fine-tuning large language models for vulnerability detection,” IEEE Access, 2025.	Utilized PEFT with LoRA for a 13B parameter model and implemented a fast batch packing strategy.	Finetuned WizardCoder outperformed CodeBERT models in ROC AUC and F1 measures on Java datasets.	Exploring curriculum learning, data augmentation, and utilizing project-level context.
[10]	K. Tamberg and H. Bahsi, “Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study,” IEEE Access, 2025.	Applied multi-class classification to 578 Java files using 25 different prompting strategies across GPT-4 and Claude 3.	LLMs outperformed traditional tools in recall and F1 scores, specifically GPT-4 with Chain of Thought prompts.	Explore LLM agents interacting with compilers and using larger context windows for multi-file analysis.
[11]	Y. Li, K. Joshi, X. Wang, and E. Wong, “Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement,” in 2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA), 2025.	A vulnerability analyst agent for tool-based cross-procedural code analysis, a security architect agent for iterative feedback, and an evaluation judge agent for fine-grained, unbiased performance assessment.	Outperforms existing single and multi-agent systems, achieving a 62% higher pairwise accuracy than other multi-agent frameworks constrained by the context window limits of large language models and the high token costs associated.	Can be extended with additional vulnerability-related databases or knowledge retrieval techniques.

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[12]	D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," IEEE Access, 2025.	Aggregated project code into prompts via GitHub Models API; results converted to SARIF for verification.	LLMs outperformed traditional tools in F1 scores but struggled with pinpointing due to hallucinations.	Exploring prompt engineering and developing hybrid pipelines combining LLMs with rule-based scanners.
[13]	S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, "Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis," IEEE Access, 2024.	Processes code similarity through a split transformer architecture consisting of a teacher encoder and a student encoder optimized via supervised contrastive learning.	XTransformer significantly outperforms state-of-the-art models in accuracy and F1 score. Reduces inference time considerably (4.09x to 9.38x faster).	Enhancing dataset diversity by incorporating a wider variety of programming languages. Integrating eXplainable AI (XAI) techniques.
[14]	S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, "Codesmash: Source-code vulnerability detection using siamese and multi-level neural architecture," IEEE Access, 2024.	A Siamese neural network named CODESMASH utilizes a hierarchical structure integrating BiGRU and attention mechanisms to analyze code similarity.	The model significantly outperforms existing methods in metrics, showing superior capability in handling lengthy, complex code sequences.	Future research will expand generalizability to various programming languages and reduce computational overhead to improve efficiency.

Paper No.	Paper Description	Proposed Methodology/ Algorithm	Key Findings	Future Scope
[15]	G. Li and Y. Yang, “On the code vulnerability detection based on deep learning: A comparative study,” IEEE Access, 2024.	Abstracting program representations from source code through lexical, syntactic, and semantic analyses combined with compiler-related techniques.	While sequence-based detection misses multi-dimensional structures, semi-synthetic datasets merge benefits to optimize robustness, driving F1 scores above 0.90.	Leverage a multi-granularity framework pairing coarse function-level filters with fine-grained statement analysis.

3 Problem Statement

The problem statement is the design and implementation of an autonomous multi-agent security auditing framework that integrates Bayesian inference across heterogeneous structural, semantic, and contextual agents to secure CI/CD pipelines during the pull request phase. The system aims to improve vulnerability detection and minimize false-positive rates by operating within isolated, tamper-resistant environments, thereby enabling precise, explainable, and automated security assessments without requiring manual intervention.

4 Proposed Methodology

The proposed system will be an agentic security auditing platform that will automate vulnerability detection and remediation across GitHub pull requests. It will combine static, semantic, and contextual code analysis, and it will use a formal Bayesian inference model to make automated security review simpler and stronger. Developers will interact with the system through GitHub, where every pull request will automatically trigger a security audit with no manual intervention needed, and they will also use a web-based dashboard that will show audit history, verdicts, and evidence. The backend will process each pull request using multiple specialist agents that will independently analyze the code, and their evidence will then be fused into a calibrated verdict by a Bayesian Inference Engine. The system will also include a patch generation and verification component that will automatically suggest fixes for confirmed vulnerabilities and then validate those fixes. It will follow a modular architecture made up of a presentation layer, a GitHub integration layer, a multi-agent analysis layer, a Bayesian fusion layer, and a patch verification layer.

4.1 System Architecture

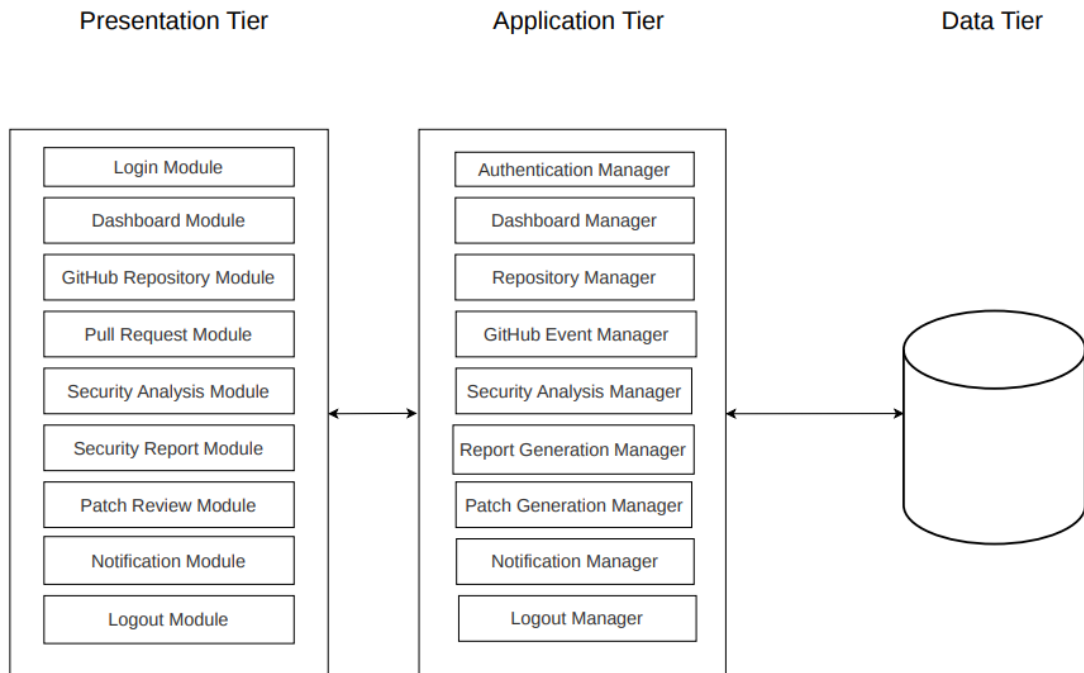


Figure 1: System Architecture of CodeSheriff

The system architecture follows a layered and modular design to ensure scalability, maintainability, and clear separation between the GitHub-facing interface, the core auditing logic, and persistent storage. Each layer performs a specific function and interacts with the adjacent layers through well-defined interfaces.

Presentation tier will be the part of CodeSheriff that users see and use directly. Users will log in through the Login Module and land on the Dashboard, which will show their repositories, audit history, and vulnerability stats. The GitHub Repository and Pull Request Modules will let users connect repos and track PR audits. The Security Analysis Module will display detected vulnerabilities with severity and confidence scores, and the Security Report Module will turn these into readable reports. The Patch Review Module will let developers review AI-suggested fixes before applying them. The Notification Module will alert users about new findings, and the Logout Module will end sessions securely. This layer will only display data it will not perform any analysis itself.

Application tier will handle all the core logic. The Authentication Manager will manage logins and sessions, and the GitHub Event Manager will listen for webhooks to trigger audits automatically when a PR opens. The Security Analysis Manager will be the core piece — it will orchestrate the four agents (Static, Semantic, Context, Runtime) and pass their evidence to the Bayesian Inference Engine to generate a verdict. The Report Generation Manager will format this into the PR comment, and the Patch Generation Manager will call GPT-4o to draft fixes. The Notification Manager will send results back to GitHub and the dashboard. This layer will essentially process events and requests, run them through the detection pipeline, and return decisions.

Data tier will store everything the system needs to persist, using PostgreSQL with Redis for caching. It will hold user data, repository metadata, PR details, and most importantly the vulnerability findings, posteriors, BUI scores, and agent evidence from every audit. It will also store generated reports, patches, and notification history. This tier will not perform any analysis itself it will simply preserve records for later review, comparison, and the research paper's benchmarking.

4.2 Tools & Technology

Technology Stack

Table 2: Technology Stack Used

Component	Version	Purpose
Python	3.11	Programming language
TypeScript / JavaScript	ES2023	Programming language
FastAPI	0.11x	Backend orchestrator
Inngest	Latest	Background jobs & event orchestration
Next.js	14	Frontend dashboard interface
Tailwind CSS	3	Frontend styling
Wasmtime	Latest	Sandbox execution runtime
Docker / Docker Compose	Latest	Containerized deployment
AWS Cloud	–	Confidential compute & hosting
Redis	7	Caching & task queue
PostgreSQL	16	Database

Software Requirements

Table 3: Software Tools Used

Component	Version	Platform
IDE	1.85	Visual Studio Code
Operating System	11	Windows
Browser	121	Chrome
Container Runtime	Latest	Docker
Version Control	2.4x	Git

Hardware Requirements

Table 4: Hardware Requirements for CodeSheriff

Component	Specification
Processor	11th Gen Intel(R) Core(TM) i5-11320H @ 3.20 GHz, 3187 MHz, 4 Core(s), 8 Logical Processor(s)
RAM	16 GB DDR4 SDRAM
Storage	512 GB SSD
GPU	NVIDIA GeForce RTX 3050, 4 GB GDDR6 VRAM

5 Project Planning and Implementation

Table 5: Project Planning Phases

Phase	Duration	Tasks
Phase 1: Planning and Requirement Analysis	15 June – 30 June	• Define project objectives, scope, and limitations • Identify agent roles: Static, Semantic, Context, and Runtime/Confidential-Execution agents • Analyze functional and non-functional requirements, including dataset and cloud infrastructure needs • Study existing multi-agent vulnerability detectors and related work
Phase 2: Methodology and System Approach	1 July – 15 July	• Define overall system workflow: GitHub PR ingestion, parallel agent dispatch, evidence fusion • Design agent communication protocol and standardized JSON evidence schema • Formalize the Bayesian Vulnerability Inference model (prior, likelihood ratios, sequential posterior update) • Design high-level system architecture and data flow across the orchestrator and agents

Phase	Duration	Tasks
Phase 3: Development and Implementation	16 July – 15 September	• Develop Static Analysis Agent and Semantic Reasoning Agent • Develop Context Agent and Patch Agent for fix suggestions • Implement Bayesian Inference Engine — sequential evidence fusion and Bayesian Uncertainty Index • Integrate GitHub App, orchestrator, and dashboard for end-to-end PR auditing • Set up the confidential execution layer as a supporting component (SFI-Wasm sandbox and Nitro Enclave environment) so the Runtime Agent can begin feeding evidence into the fusion pipeline
Phase 4: Design and Architecture	16 September – 5 October	• Perform unit and integration testing of all four agents and the orchestrator • Validate Bayesian fusion correctness, including handling of missing evidence • Evaluate detection performance on BigVul/CVEfixes and run ablation studies across agent combinations • Identify and fix functional and logical issues across the pipeline
Phase 5: Testing, Deployment, and Evaluation	6 October – 20 October	• Deploy the system end-to-end, including the SFI-Enabled Confidential CI/CD Execution Layer • Run security evaluation: attestation success rate, SFI containment under fault injection, TCB-size comparison • Collect calibration and detection results; analyze and document findings • Prepare final project report, research paper draft, and presentation

6 Expected Outcome

By the end of the project development cycle, the following outcomes are expected:

- **Intelligent multi-agent detection module**

This module will analyze pull requests from multiple complementary perspectives structural, semantic, and contextual to identify security vulnerabilities more comprehensively than a single-method approach. The agents will act as independent specialists, ensuring vulnerabilities will be caught even when they are visible from only one angle.

- **Evidence fusion mechanism**

This mechanism will combine the findings of all agents into a single, calibrated verdict along with a measure of confidence in that verdict. It will replace simple voting with a more principled, statistically grounded approach, improving the reliability and trustworthiness of the system's decisions.

- **Secure execution environment**

This environment will safely run pull request code in isolation, protecting sensitive build credentials from exposure even in the event of a compromised or malicious pull request. It will ensure that the system not only detects vulnerabilities but also actively safeguards the development pipeline itself.

- **Deployed system**

This system will be integrated directly into the GitHub pull request workflow, along with a web-based dashboard, and will automatically provide developers with clear vulnerability explanations, severity ratings, and suggested fixes. It will support end-to-end detection, explanation, and patch verification, making automated security review accessible without disrupting existing developer workflows.

7 Conclusion

In this synopsis, we conclude that CodeSheriff, an Agentic Security Auditor for GitHub pull requests, offers a structured and practical way to strengthen CI/CD security through intelligent, multi-modal vulnerability detection. Through this project, we moved beyond single-method scanning by combining structural, semantic, and contextual analysis, and we fused their evidence through a formal Bayesian inference model to produce calibrated, trustworthy verdicts. We used a methodology built on a modular multi-agent architecture, confidential execution for runtime evidence, and automated patch generation and verification to ensure accurate and reliable security auditing. We conclude that the system reduces false positives, improves detection coverage, and protects the CI/CD pipeline itself against compromise. Overall, this synopsis establishes a clear framework, set of objectives, and implementation strategy that demonstrate the feasibility and academic relevance of the project.

References

- [1] P.-C. Huang, C.-H. Mao, and C.-M. Lai, “Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification,” *IEEE Access*, 2026.
- [2] R. A. Almutairi and N. Abbadeni, “Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code,” *IEEE Access*, 2026.
- [3] G. Antal, L. Értekes, N. Szolnoki, and P. Hegedűs, “Evaluating retrieval-augmented generation for llm-based vulnerability detection: An empirical study on real-world java vulnerabilities,” *IEEE Access*, 2026.
- [4] B. K. E. Abbasi and A. Shameli-Sendi, “Leveraging key source code metrics for predicting software vulnerabilities with machine learning,” *IEEE Access*, 2026.
- [5] H. Gao, D. Zhang, and L. Chen, “Scout: Llm-driven vulnerability detection via interactive repository exploration,” *IEEE Access*, 2026.
- [6] K. P. Reddy, J. S. Dhatteval *et al.*, “Agentic artificial intelligence with knowledge-based reasoning for automated vulnerability triage in collaborative software repositories,” in *2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies (IC2PCT)*. IEEE, 2026, pp. 493–498.
- [7] L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, “A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models,” *IEEE Access*, 2025.
- [8] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, “Advanced smart contract vulnerability detection via llm-powered multi-agent systems,” *IEEE Transactions on Software Engineering*, 2025.
- [9] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, “Finetuning large language models for vulnerability detection,” *IEEE Access*, 2025.
- [10] K. Tamberg and H. Bahsi, “Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study,” *IEEE Access*, 2025.
- [11] Y. Li, K. Joshi, X. Wang, and E. Wong, “Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement,” in *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2025.
- [12] D. Gnieciak and T. Szandala, “Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection,” *IEEE Access*, 2025.

- [13] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, "Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis," *IEEE Access*, 2024.
- [14] S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, "Code-smash: Source-code vulnerability detection using siamese and multi-level neural architecture," *IEEE Access*, 2024.
- [15] G. Li and Y. Yang, "On the code vulnerability detection based on deep learning: A comparative study," *IEEE Access*, 2024.
- [16] B. Gülmez, "Code generation with large language models: a survey from neural program synthesis to autonomous software development," *Applied Intelligence*, vol. 56, no. 6, p. 200, 2026.
- [17] H. Su, Z. Xu, Y. Zhang, and Q. Tan, "Source code vulnerability detection based on deep learning: a review," *Cybersecurity*, vol. 9, no. 1, p. 2, 2026.
- [18] Y. Chen, Y. Shen, T. Wang, S. Ou, R. Wang, Y. Li, and Z. Pan, "Software defect detection using large language models: a literature review," *Frontiers of Computer Science*, vol. 20, no. 6, p. 2006202, 2026.
- [19] T. A. Syed, M. Riyaz Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, "Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation," in *2025 International Conference on Computer and Applications (ICCA)*, 2025.
- [20] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, "Enhanced code reviews using pull request based change impact analysis," *Empirical Software Engineering*, vol. 30, no. 3, p. 64, 2025.
- [21] Amazon Web Services, AWS Nitro Enclaves Documentation Team, "AWS Nitro Enclaves user guide," <https://docs.aws.amazon.com/enclaves/latest/user/>, 2026, accessed: 2026-07-24.
- [22] Amazon Web Services, AWS Nitro Enclaves Concepts Guide, "AWS Nitro Enclaves, concepts," <https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave.html>, 2026, accessed: 2026-07-24.
- [23] Amazon Web Services, AWS Nitro Enclaves Attestation Guide, "Cryptographic attestation in AWS Nitro Enclaves," <https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave-concepts.html#term-attestdoc>, 2026, accessed: 2026-07-24.
- [24] Amazon Web Services, AWS Nitro Enclaves CLI Team, "AWS Nitro Enclaves CLI (nitro-cli)," <https://github.com/aws/aws-nitro-enclaves-cli>, 2026, gitHub repository, accessed: 2026-07-24.

- [25] Amazon Web Services, AWS Nitro Enclaves SDK Team, “AWS Nitro Enclaves SDK (NSM API),” <https://github.com/aws/aws-nitro-enclaves-sdk-c>, 2026, gitHub repository, accessed: 2026-07-24.
- [26] Amazon Web Services, Security Whitepapers Team, “The security design of the AWS nitro system,” <https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/security-design-of-aws-nitro-system.html>, 2026, aWS Whitepaper, accessed: 2026-07-24.
- [27] Amazon Web Services, AWS KMS Documentation Team, “AWS Key Management Service developer guide,” <https://docs.aws.amazon.com/kms/latest/developerguide/>, 2026, accessed: 2026-07-24.
- [28] Amazon Web Services, AWS KMS Nitro Integration Team, “Attestation-based key policies for AWS Nitro Enclaves,” <https://docs.aws.amazon.com/kms/latest/developerguide/services-nitro-enclaves.html>, 2026, accessed: 2026-07-24.
- [29] Amazon Web Services, AWS Secrets Manager Documentation Team, “AWS Secrets Manager user guide,” <https://docs.aws.amazon.com/secretsmanager/latest/userguide/>, 2026, accessed: 2026-07-24.
- [30] Amazon Web Services, EC2 Documentation Team, “Amazon EC2 nitro-based instances,” <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-types.html#ec2-nitro-instances>, 2026, accessed: 2026-07-24.
- [31] Bytecode Alliance, Wasmtime Documentation Team, “Wasmtime documentation,” <https://docs.wasmtime.dev/>, 2026, accessed: 2026-07-24.
- [32] Bytecode Alliance, Wasmtime API Reference Team, “Wasmtime API reference, Config::consume_fuel,” https://docs.rs/wasmtime/latest/wasmtime/struct.Config.html#method.consume_fuel, 2026, accessed: 2026-07-24.
- [33] WasmEdge Community, “WasmEdge runtime documentation,” <https://wasmedge.org/docs/>, 2026, accessed: 2026-07-24.
- [34] WASI Subgroup, W3C WebAssembly Community Group, “WASI: WebAssembly system interface specification,” <https://wasi.dev/>, 2026, accessed: 2026-07-24.
- [35] WebAssembly Community Group, W3C, “WebAssembly specification (core), release 2.0,” <https://webassembly.github.io/spec/core/>, 2026, accessed: 2026-07-24.
- [36] Bytecode Alliance, Javy Maintainers, “Javy: A JavaScript-to-WebAssembly toolchain,” <https://github.com/bytecodealliance/javy>, 2026, gitHub repository, accessed: 2026-07-24.

- [37] S. Cabrera, “Bringing JavaScript to WebAssembly for Shopify functions,” Shopify Engineering Blog, Feb. 2023. [Online]. Available: <https://shopify.engineering/javascript-in-webassembly-for-shopify-functions>
- [38] Bytecode Alliance, Technical Steering Committee, “Welcoming Javy: A new hosted project,” Bytecode Alliance Blog, Jun. 2023. [Online]. Available: <https://bytecodealliance.org/articles/javy-hosted-project>
- [39] Tree-sitter Maintainers, “Tree-sitter documentation,” <https://tree-sitter.github.io/tree-sitter/>, 2026, accessed: 2026-07-24.
- [40] PyTorch Geometric Team, “PyTorch Geometric documentation,” <https://pytorch-geometric.readthedocs.io/>, 2026, accessed: 2026-07-24.
- [41] PyTorch Geometric API Reference Team, “SAGEConv API reference,” https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.SAGEConv.html, 2026, accessed: 2026-07-24.
- [42] Hugging Face, Transformers Documentation Team, “Transformers documentation,” <https://huggingface.co/docs/transformers/>, 2026, accessed: 2026-07-24.
- [43] Microsoft, CodeBERT Model Card, “CodeBERT-base model card,” <https://huggingface.co/microsoft/codebert-base>, 2026, hugging Face Hub, accessed: 2026-07-24.
- [44] Meta AI / Facebook Research, Faiss Wiki, “Faiss: A library for efficient similarity search,” <https://github.com/facebookresearch/faiss/wiki>, 2026, accessed: 2026-07-24.
- [45] UKP Lab, Sentence-Transformers Documentation, “Sentence-transformers documentation,” <https://www.sbert.net/>, 2026, accessed: 2026-07-24.
- [46] OpenAI, API Reference Team, “OpenAI API reference,” <https://platform.openai.com/docs/>, 2026, accessed: 2026-07-24.
- [47] National Institute of Standards and Technology (NIST), NVD API Team, “National Vulnerability Database (NVD) API documentation,” <https://nvd.nist.gov/developers/vulnerabilities>, 2026, accessed: 2026-07-24.
- [48] NumPy Developers, “NumPy documentation,” <https://numpy.org/doc/>, 2026, accessed: 2026-07-24.
- [49] SciPy Developers, “SciPy documentation, scipy.stats,” <https://docs.scipy.org/doc/scipy/reference/stats.html>, 2026, accessed: 2026-07-24.
- [50] PyMC Developers, “PyMC documentation,” <https://www.pymc.io/>, 2026, accessed: 2026-07-24.

- [51] NumPyro Developers, “NumPyro documentation,” <https://num.pyro.ai/>, 2026, accessed: 2026-07-24.
- [52] FastAPI Maintainers, “FastAPI documentation,” <https://fastapi.tiangolo.com/>, 2026, accessed: 2026-07-24.
- [53] Celery Project, “Celery, distributed task queue documentation,” <https://docs.celeryq.dev/>, 2026, accessed: 2026-07-24.
- [54] Redis Ltd., Documentation Team, “Redis documentation,” <https://redis.io/docs/latest/>, 2026, accessed: 2026-07-24.
- [55] PostgreSQL Global Development Group, “PostgreSQL documentation,” <https://www.postgresql.org/docs/>, 2026, accessed: 2026-07-24.
- [56] GitHub, Inc., GitHub Apps Documentation Team, “Building GitHub Apps documentation,” <https://docs.github.com/en/apps/creating-github-apps>, 2026, accessed: 2026-07-24.
- [57] GitHub, Inc., Octokit Maintainers, “Octokit.js, GitHub REST/GraphQL API client,” <https://github.com/octokit/octokit.js>, 2026, gitHub repository, accessed: 2026-07-24.
- [58] GitHub, Inc., Webhooks Documentation Team, “Webhooks documentation,” <https://docs.github.com/en/webhooks>, 2026, accessed: 2026-07-24.
- [59] GitHub, Inc., GitHub Actions Documentation Team, “GitHub Actions documentation,” <https://docs.github.com/en/actions>, 2026, accessed: 2026-07-24.
- [60] GitHub, Inc., OAuth Apps Documentation Team, “Building OAuth apps documentation,” <https://docs.github.com/en/apps/oauth-apps>, 2026, accessed: 2026-07-24.
- [61] Docker Inc., Compose Documentation Team, “Docker Compose documentation,” <https://docs.docker.com/compose/>, 2026, accessed: 2026-07-24.
- [62] ngrok, Inc., “ngrok documentation,” <https://ngrok.com/docs>, 2026, accessed: 2026-07-24.
- [63] Meta, React Documentation Team, “React documentation,” <https://react.dev/>, 2026, accessed: 2026-07-24.
- [64] Vite Maintainers, “Vite documentation,” <https://vite.dev/>, 2026, accessed: 2026-07-24.
- [65] Recharts Maintainers, “Recharts documentation,” <https://recharts.org/>, 2026, accessed: 2026-07-24.
- [66] Tailwind Labs, “Tailwind CSS documentation,” <https://tailwindcss.com/docs>, 2026, accessed: 2026-07-24.

- [67] Sentry, Documentation Team, “Sentry documentation,” <https://docs.sentry.io/>, 2026, accessed: 2026-07-24.
- [68] MITRE Corporation, CWE Program, “Common weakness enumeration (CWE) list,” <https://cwe.mitre.org/>, 2026, accessed: 2026-07-24.
- [69] MITRE Corporation and NIST, CVE Program, “Common vulnerabilities and exposures (CVE) program,” <https://www.cve.org/>, 2026, accessed: 2026-07-24.
- [70] OWASP Foundation, “OWASP top 10,” <https://owasp.org/www-project-top-ten/>, 2026, accessed: 2026-07-24.
- [71] Trail of Bits, Slither Maintainers, “Slither, static analysis framework,” <https://github.com/cryptic/slither>, 2026, gitHub repository, accessed: 2026-07-24.
- [72] SonarSource, SonarQube Documentation Team, “SonarQube community edition documentation,” <https://docs.sonarsource.com/sonarqube/>, 2026, accessed: 2026-07-24.
- [73] Google, gVisor Documentation Team, “gVisor documentation,” <https://gvisor.dev/docs/>, 2026, accessed: 2026-07-24.
- [74] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [75] W. L. Hamilton, *Graph Representation Learning*, ser. Synthesis Lectures on Artificial Intelligence and Machine Learning. San Rafael, CA, USA: Morgan & Claypool Publishers, 2020.
- [76] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [77] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2013.
- [78] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2022.
- [79] E. T. Jaynes, *Probability Theory: The Logic of Science*. Cambridge, UK: Cambridge University Press, 2003.
- [80] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1994.
- [81] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Indianapolis, IN, USA: Wiley, 2020.

- [82] M. Bishop, *Computer Security: Art and Science*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [83] N. Ferguson, B. Schneier, and T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*. Indianapolis, IN, USA: Wiley, 2010.
- [84] G. McGraw, *Software Security: Building Security In*. Boston, MA, USA: Addison-Wesley, 2006.
- [85] V. Costan and S. Devadas, “Intel SGX explained,” IACR Cryptology ePrint Archive, Report 2016/086, 2016. [Online]. Available: <https://eprint.iacr.org/2016/086>
- [86] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [87] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2014.
- [88] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512.
- [89] G. P. Bhandari, A. Naseer, and L. Moonen, “CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software,” in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*, 2021, pp. 30–39.
- [90] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics (EMNLP)*, 2020, arXiv preprint arXiv:2002.08155.

Mr. Nirmalya Mukhopadhyay

Asst. Prof., Dept. of IT

Project Guide

Mr. Kunal Kundu

Asst. Prof., Dept. of IT

Project Guide

Mrs. Kiran Patil

Project Coordinator

Dr. Poonam Gupta

Head of Department